Python Engineering

Exercise 05 - Loops: Sum of Squares

Theory + mental model + worked trace + tests + learning tracker

Exercise objective: understand iteration through four core ideas: counter, accumulator, state and loop, and connect them with the future numerical integration of engineering dynamics.

1. The mathematical problem

We want to compute the sum of the squares of all integers from 1 to N:

$$S = \sum_{i=1}^{N} i^2$$

For N = 4, the calculation is:

$$S = 1^2 + 2^2 + 3^2 + 4^2 = 30$$

In this exercise we deliberately do not use the closed-form expression. The goal is to make the computer perform the repeated operation explicitly.

> **Key idea: when the same operation must be applied to a sequence of values, an iteration naturally appears.**

2. What changes compared with previous exercises?

Previously, most programs followed a short path: input → calculation → output. Here, one operation is repeated for 1, 2, 3, ..., N.

```
1^2
2^2
3^2
4^2
...
N^2
```

A loop needs a way to know which element is being processed. In this exercise that role is played by i.

The counter

The variable i changes at each pass through the loop: 1, 2, 3, ..., N. It is the counter or current iteration index.

3. Accumulator and program state

The program must remember the partial sum from previous iterations. We therefore initialize a variable, for example total = 0, and update it at every pass.

$$total_{new} = total_{old} + i^2$$

This variable is called an accumulator. Its value contains information produced by earlier iterations.

#	i	total
initial	—	0
1	1	1
2	2	5
3	3	14
4	4	30

State means the current information stored by the algorithm. Later, the same idea will appear as x(k+1) = f(x(k)) in numerical simulation.

Connection with engineering dynamics

The sum-of-squares exercise looks simple, but its update pattern is structurally similar to a time-marching simulation:

$$v_{k+1} = v_k + a_k \, \Delta t$$

$$h_{k+1} = h_k + v_k \, \Delta t$$

In both cases the next state is built from the previous state.

4. Choosing a loop: for vs while

Here we know in advance how many values must be processed: 1 through N. Therefore a for loop is the natural choice.

```
Known number of iterations  ->  for
Stopping condition           ->  while
```

This is not an absolute rule, but it is a useful first mental model.

The range() trap

Python excludes the upper bound of range. Therefore range(1, N) produces 1, 2, ..., N-1, not N.

```
print(list(range(1, N)))
```

During development, printing a range as a list is a quick way to verify exactly which indices will be visited.

5. Algorithm before code

```
Define N
Validate N
Initialize accumulator

FOR each integer from 1 to N:
    calculate square
    add square to accumulator

Display accumulator
```

Do not add branches or abstractions that the problem does not require. The goal of EX05 is to understand the loop itself.

6. Trace the loop while it runs

Temporarily print the counter and accumulator inside the loop. This is a simple form of trace debugging.

```
i = 1 | total = 1
i = 2 | total = 5
i = 3 | total = 14
i = 4 | total = 30
```

Use the VS Code debugger as well: place a breakpoint inside the loop and press F10 repeatedly. Watch i and total change in the Variables panel.

Assignment is not mathematical equality

A statement such as total = total + i**2 does not mean total equals itself plus something. It means: read the old value, compute a new one, and store it back.

$$total_{new} = total_{old} + i^2$$

Python also offers the compact form total += i**2, which expresses the same update.

7. Validate the iterative result

After the loop, compare the result with the analytical expression. We use the formula only as a reference solution:

$$S_{exact} = N(N+1)(2N+1) / 6$$

The important engineering habit is: numerical/algorithmic result vs known reference.

Required test cases

N	Expected S	Purpose
1	1	minimum normal case
4	30	manual trace
10	385	compare with formula
0	policy to decide	boundary case
-5	reject	invalid input

8. Learning tracker - EX05

Concept	What should be understood after EX05
Iteration	Repeat the same operation over a sequence of values.
Counter	Know which iteration/value is currently being processed.
Accumulator	Store a partial result that evolves during the loop.
State	Understand that the current value depends on previous iterations.
for vs while	Use for naturally when the number of passes is known; while when a stopping condition drives the loop.
range()	Remember that the upper bound is excluded.
Debug trace	Observe how i and total evolve at each iteration.
Validation	Compare the algorithmic result with an analytical reference.

Next step: EX06 changes the accumulator from additive to multiplicative by implementing N!.